Enterprise Integration Architecture for Core Banking Modernization

Global Tier-1 Banking Solution

Prepared for: DevSu
Prepared by: Reinaldo Otálvaro
Date: March 6, 2026
Version: 3.0

Table of Contents

1. **Executive Summary**
    - 1.1 The Challenge
    - 1.2 The Solution
    - 1.3 Key Differentiators
2. **Architectural Vision & Guiding Principles**
    - 2.1 Vision Statement
    - 2.2 Design Principles
    - 2.3 Standards Alignment (BIAN, ISO 20022)
3. **C4 Model Diagrams**
    - 3.1 Level 1: System Context Diagram
    - 3.2 Level 2: Container Diagram
    - 3.3 Level 3: Component Diagram
    - 3.4 Level 4: Code Diagram
4. **Enterprise Architecture Diagrams**
    - 4.1 Security Architecture Diagram
    - 4.2 High Availability & Disaster Recovery Diagram
    - 4.3 Event Streaming & Data Flow Diagram
    - 4.4 Migration Roadmap (Strangler Pattern)
5. **Technology Realization Decisions**
    - 5.1 Platform Selection Matrix
    - 5.2 Technology Justification Framework
6. **Governance & Operating Model**
    - 6.1 API Governance Framework
    - 6.2 Center for Enablement (C4E) Model
    - 6.3 Microservices Governance
7. **Compliance & Regulatory Framework**
    - 7.1 Global Regulatory Mapping
    - 7.2 Audit Architecture

- 7.3 Data Privacy & Protection

8. **Appendix**
    - 8.1 BIAN Service Domain Mapping
    - 8.2 API Performance SLAs
    - 8.3 Disaster Recovery Playbook
    - 8.4 Glossary

1. Executive Summary

1.1 The Challenge

A traditional bank requires a comprehensive integration architecture to modernize its digital services, integrate multicore banking environments, and comply with emerging regulations while ensuring high availability, security, and low latency. The bank must integrate:

- Traditional core banking + new digital core
- New web and mobile banking
- Payment services platform
- APIs for Open Finance
- Risk management system
- Fraud prevention system

1.2 The Solution

We present a **Cloud-Native Hybrid Integration Platform** built on **Amazon Web Services (AWS)** that establishes a robust, scalable, and secure foundation for the bank's digital transformation journey . The architecture leverages:

- **Event-Driven Architecture (EDA)** with **Apache Kafka** as the nervous system
- **API-led connectivity** through **AWS API Gateway**
- **Strangler Fig pattern** for gradual core modernization
- **BIAN-aligned** service definitions
- **ISO 20022** payment standards

1.3 Key Differentiators

Differentiator	Description	Banking Impact
BIAN-Aligned API Strategy	All services modeled after Banking Industry Architecture Network standards	Reduced integration friction; faster time-to-market for new products
Active-Active Multi-Region Deployment	True high availability with RPO < 1 second, RTO < 5 minutes	99.99% availability; regulatory compliance for business continuity
Zero-Trust Security Model	End-to-end encryption, mTLS, fine-grained authorization	PCI-DSS, GDPR, SOX compliance
Event-First Architecture	Real-time fraud detection, risk management, auditability	Millisecond fraud response; regulatory audit trails
Proven Technology Stack	AWS + Kafka + Kubernetes + Spring Boot	Used by global Tier-1 banks

2. Architectural Vision & Guiding Principles

2.1 Vision Statement

"To create a future-proof integration backbone that enables business agility, ensures regulatory compliance, and delivers exceptional customer experiences through seamless, secure, and real-time connectivity across all banking systems."

2.2 Design Principles

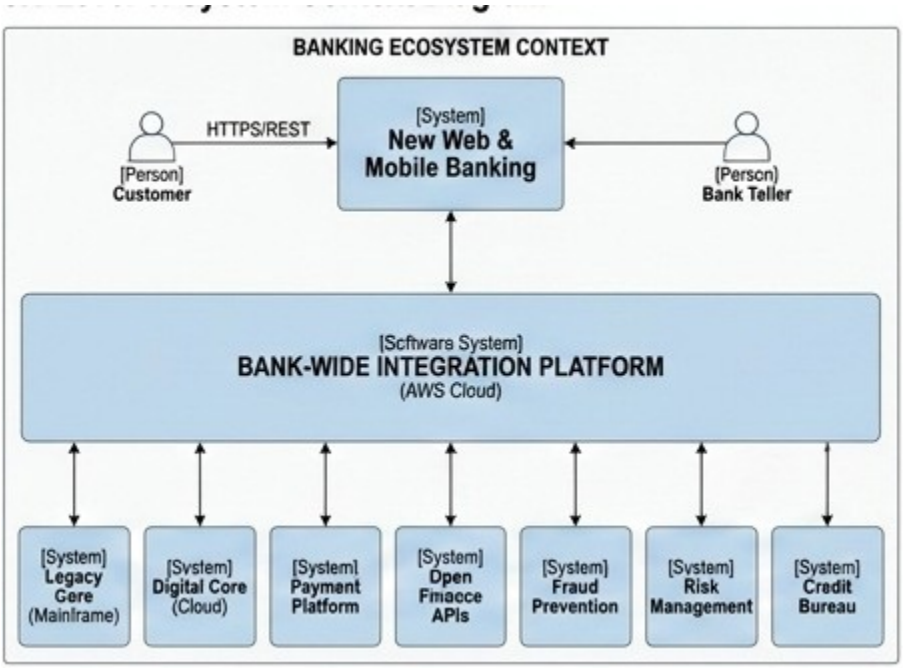
Principle	Description	Architectural Impact
Resilience by Default	No single point of failure; self-healing systems	Multi-AZ, multi-region deployment; circuit breakers; retry with exponential backoff
Security Embedded	Security controls at every layer, not just perimeter	Zero-trust network; mTLS; KMS encryption; WAF; fine-grained IAM
Loose Coupling	Systems communicate through well-defined interfaces	Event-driven architecture; API facade; canonical data models (BIAN)
Observability First	Everything measured, traced, logged	Distributed tracing (X-Ray); centralized logging (OpenSearch); business metrics
Automate Everything	Infrastructure, deployment, testing fully automated	Infrastructure as Code (Terraform); CI/CD pipelines (GitHub Actions)
Compliance as Code	Regulatory requirements encoded and enforced automatically	Policy as Code (Open Policy Agent); automated audit trails

2.3 Standards Alignment

Standard	Application	Benefit
BIAN	Service domain modeling; API contracts	Common language across banking domains; reduced integration complexity
ISO 20022	Payment message formats; event schemas	Data-rich payments; cross-border interoperability; regulatory readiness
OpenAPI 3.0	API specifications	Industry-standard API documentation; code generation
AsyncAPI	Event specifications	Event contract governance; schema evolution management
OAuth 2.0 / OIDC	Authentication & authorization	Industry-standard security; federated identity
CloudEvents	Event envelope standardization	Interoperability across event producers/consumers

3. C4 Model Diagrams

3.1 Level 1: System Context Diagram



Detailed Explanation:

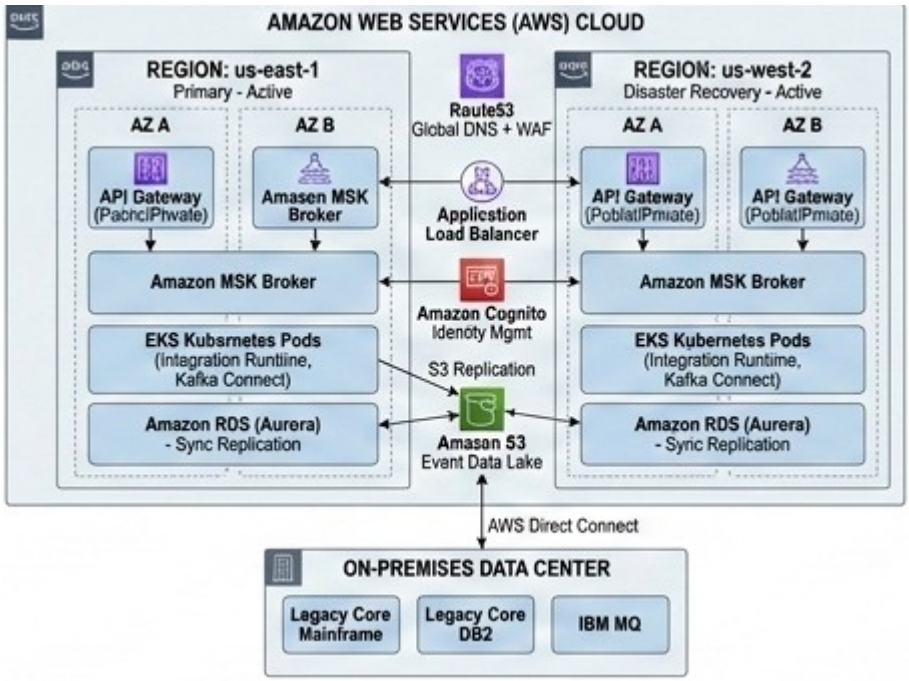
Element	Type	Description
Bank-Wide Integration Platform	Core System	Cloud-based integration hub mediating all communication between channels and systems
Customer	Person	End-user accessing banking services through digital channels
Bank Teller	Person	Internal user accessing systems through branch applications
New Web & Mobile Banking	System	Modern digital channels requiring real-time access
Legacy Core Banking	System	Existing mainframe-based core (COBOL, DB2, IMS) - requires modernization
New Digital Core Banking	System	Modern cloud-native core for new products and migrated capabilities
Payment Services Platform	System	ACH, SWIFT, cards, real-time payments
Open Finance APIs	System	External third-party APIs for open banking compliance
Fraud Prevention System	System	Real-time transaction monitoring and anomaly detection
Risk Management System	System	Enterprise-wide risk calculation

Element	Type	Description
Credit Bureau	System	External credit scoring and reporting

Key Interactions:

- All channel traffic flows **exclusively through the integration platform**
- Platform **orchestrates multicore operations** transparently to consumers
- **Event streams** feed fraud and risk systems in real-time
- **Open Finance** interactions mediated with consent management

3.2 Level 2: Container Diagram



Container Details:

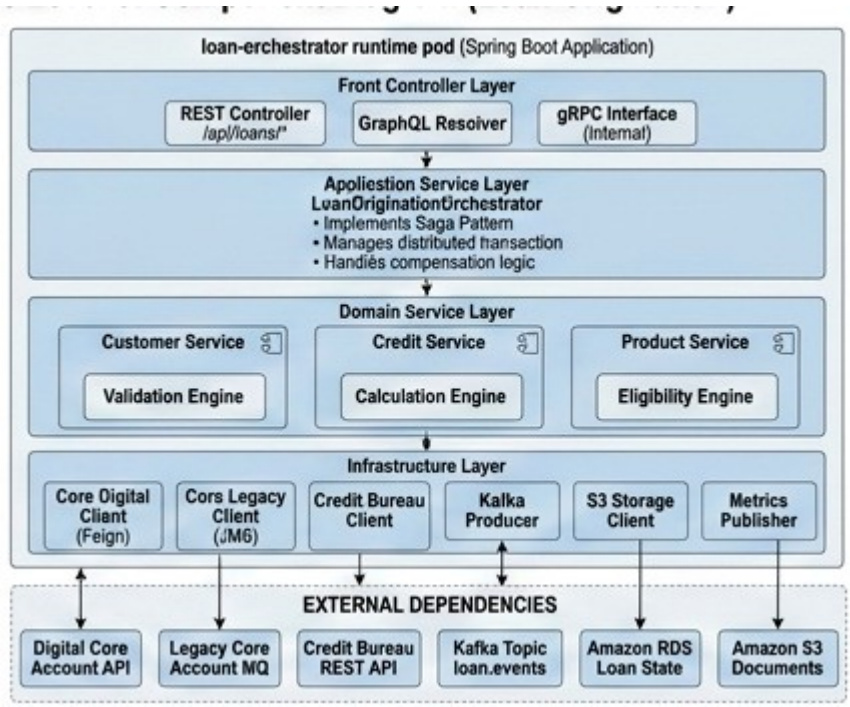
Container	Technology	Purpose	Scaling	Persistence
API Gateway	AWS API Gateway	Unified entry point for all API traffic; rate limiting; API versioning	Horizontal across AZs	Stateless
Web Application Firewall	AWS WAF	OWASP Top 10 protection; SQL injection; XSS	Global	Stateless

Container	Technology	Purpose	Scaling	Persistence
Message Broker	Amazon MSK (Kafka)	Event backbone; decouples producers/consumers; exactly-once semantics	Multi-AZ, multi-region	Persistent (7-30 days)
Integration Runtime	EKS + Spring Boot	Executes integration logic; orchestrates multicore flows	Horizontal (K8s HPA)	Stateless
Data Streaming	Kafka Connect	CDC from databases; streaming to/from external systems	Horizontal workers	Stateless
Identity Provider	Amazon Cognito	Customer identity; MFA; social login federation	Serverless	User directory
Directory Services	AWS Directory Service	Employee AD integration; SSO	Regional	Managed AD
Key Management	AWS KMS	Encryption key storage; FIPS 140-2 HSM-backed	Regional	HSM-backed
Operational DB	Amazon RDS Aurora	Platform metadata; orchestration state; consent storage	Multi-AZ, Global	SQL
Data Lake	Amazon S3	Immutable event storage; audit logs; analytics	Global	Object storage
Monitoring	CloudWatch + X-Ray	Metrics, logs, distributed tracing	Regional	30-90 days
Log Analytics	OpenSearch Service	Log aggregation; security analytics; dashboards	Regional	Indexed data

Communication Patterns:

- **Synchronous (REST):** API Gateway → Integration Runtime → Core Systems (real-time queries)
- **Asynchronous (Events):** Core Systems → Kafka → Fraud/Risk/Analytics
- **Streaming (CDC):** Databases → Kafka Connect → Data Lake

3.3 Level 3: Component Diagram (Loan Origination)



Loan Origination Flow:

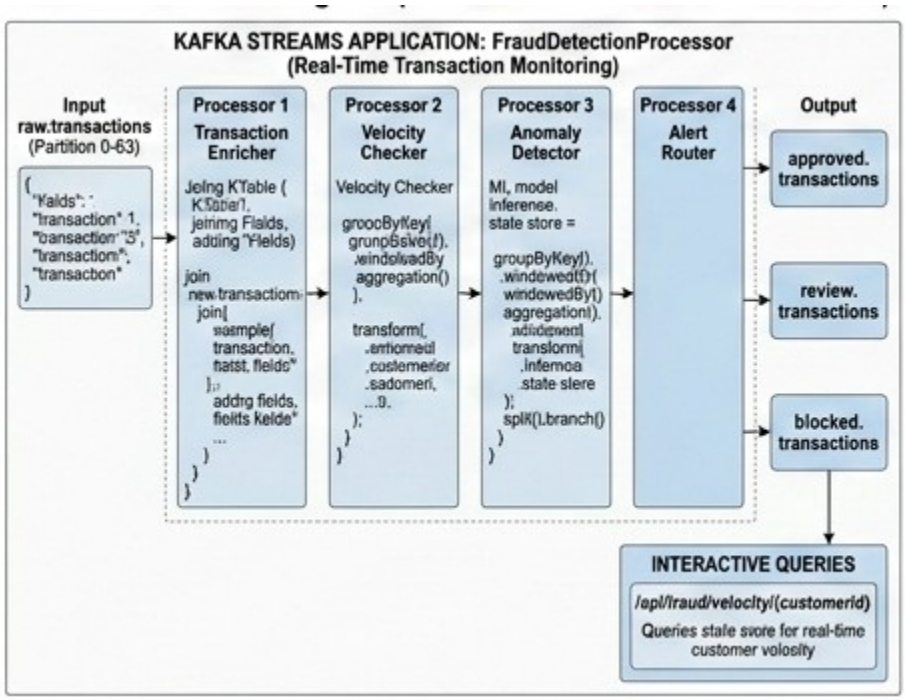
Step	Component	Action	Pattern
1	REST Controller	Receives `POST /api/loans/application`	HTTP/JSON
2	LoanOriginationOrchestrator	Initiates Saga; creates loan record	JDBC to RDS
3	CustomerService	Invokes Core Digital Client	REST/HTTPS
4	CreditService	Invokes Credit Bureau Client	SOAP/XML + mTLS
5	ProductService	Determines eligible products	In-memory
6	Orchestrator	Commits Saga on success	-

Step	Component	Action	Pattern
7	Orchestrator	Executes compensation on failure	-
8	Kafka Producer	Publishes `LoanApplicationProcessed`	Kafka
9	S3 Client	Stores loan documents	S3 PutObject
10	Metrics Publisher	Records business metrics	CloudWatch

Saga Pattern:

- **Success Path:** Customer Validated → Credit Pulled → Product Assigned → Loan Created → Event Published
- **Compensation:** If Credit Pull fails, revert Customer Validation

3.4 Level 4: Code Diagram (Fraud Detection with Kafka Streams)



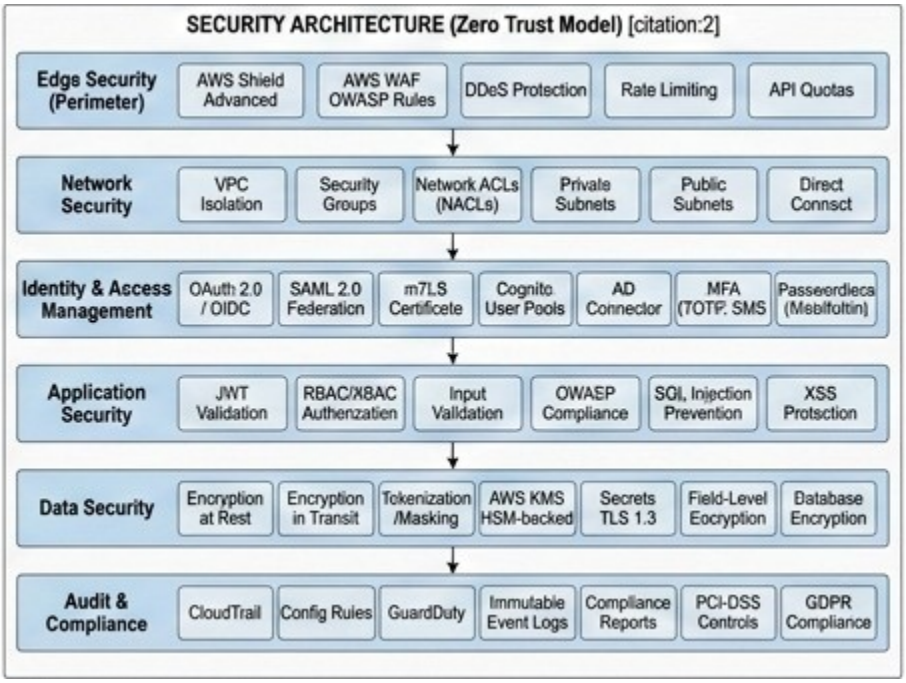
Key Benefits of Kafka Streams for Fraud Detection :

- **Exactly-Once Processing:** Guarantees no transaction is missed or double-processed
- **Millisecond Latency:** Real-time fraud detection before settlement
- **Stateful Processing:** Windowed aggregations for velocity checks
- **Fault Tolerance:** Automatic recovery with exactly-once semantics

- **Scalability:** Linear scaling with partitions

4. Enterprise Architecture Diagrams

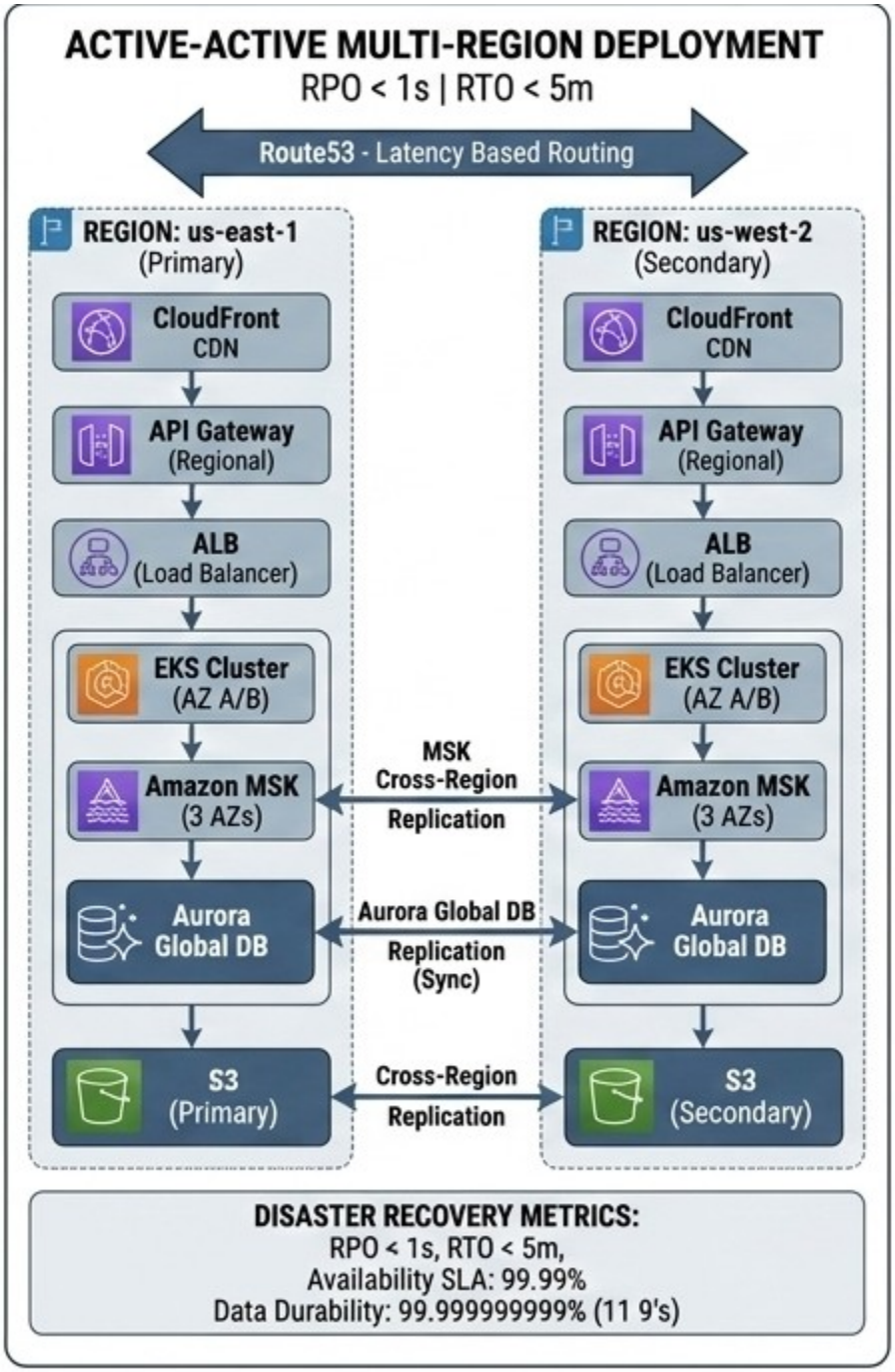
4.1 Security Architecture Diagram (Defense in Depth)



Security Controls by Layer:

Layer	Controls	Banking-Specific Implementation
Edge	DDoS protection, WAF, rate limiting	Prevents account takeover, credential stuffing
Network	VPC isolation, security groups, private subnets	Core systems never exposed to internet
IAM	OAuth 2.0/OIDC, MFA, federation	Strong authentication; SSO for employees
Application	JWT validation, RBAC/ABAC, input validation	Fine-grained authorization per banking role
Data	Encryption at rest/in transit, tokenization	PCI-DSS for card data; GDPR for PII
Audit	Immutable logs, compliance monitoring	SOX compliance; regulatory audit readiness

4.2 High Availability & Disaster Recovery Diagram

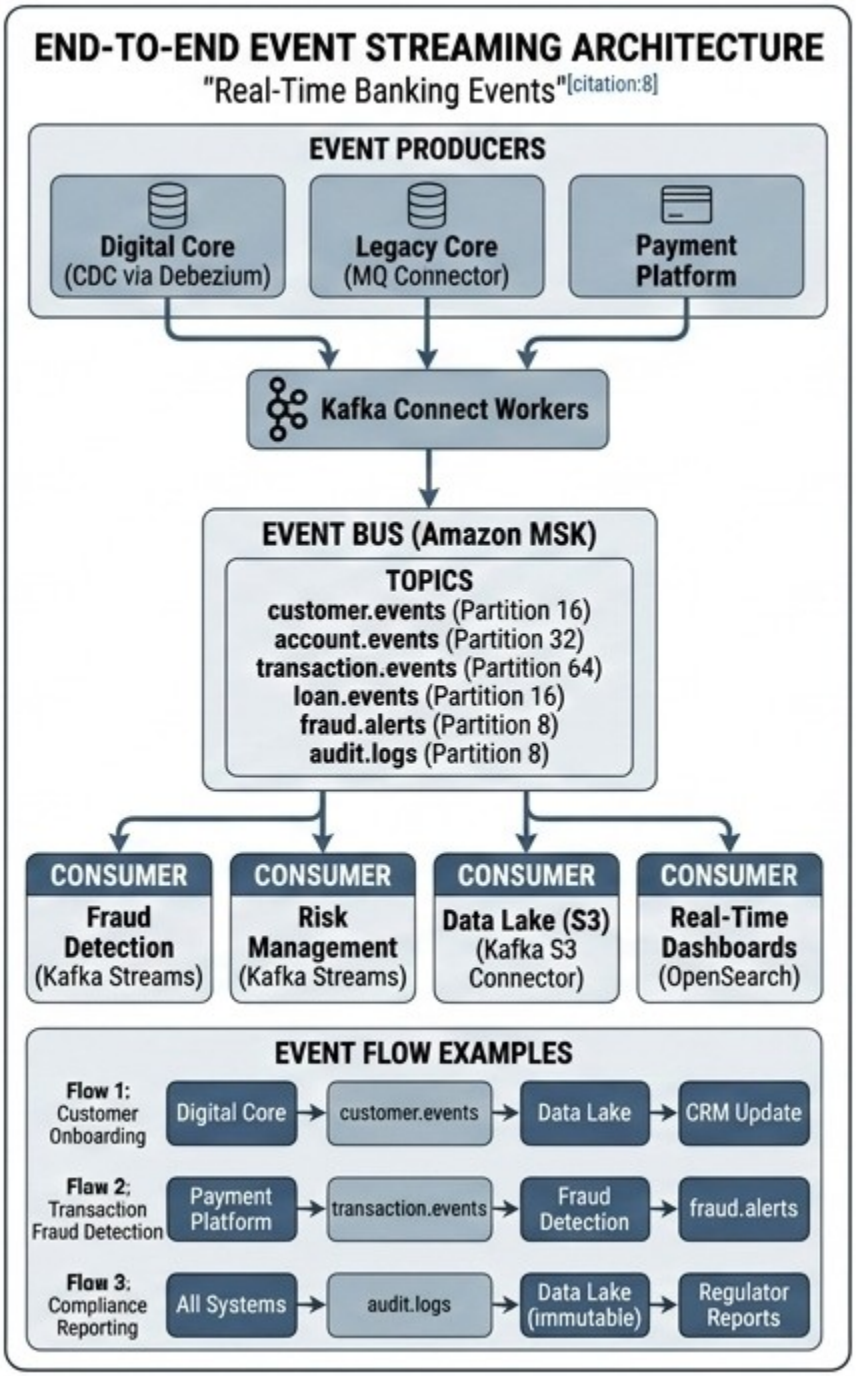


HA/DR Strategy:

Component	HA Mechanism	Failure Scenario	Recovery
DNS	Route53 health + latency routing	Region outage	Automatic traffic shift
API Gateway	Regional across AZs	AZ failure	Traffic to remaining AZs

Component	HA Mechanism	Failure Scenario	Recovery
EKS	Pods across AZs; HPA	Node failure	Kubernetes reschedules
MSK (Kafka)	3 AZs; rack awareness	Broker failure	Leader re-election
Aurora	Multi-AZ + Global DB	Region failure	Promote secondary
S3	Cross-region replication	Region failure	Failover to secondary
Direct Connect	Multiple connections	Link failure	Failover to backup

4.3 Event Streaming & Data Flow Diagram



Event Topics & Retention:

Topic	Partitions	Retention	Key	Description
customer.events	16	30 days	customerId	Profile changes, onboarding, KYC
account.events	32	30 days	accountId	Opening, closure, balance changes
transaction.events	64	7 days	transactionId	Financial transactions (high volume)
loan.events	16	90 days	loanId	Applications, approvals, disbursements
fraud.alerts	8	90 days	customerId	Fraud detection alerts
audit.logs	8	7 years	timestamp	Immutable audit trail

4.4 Migration Roadmap (Strangler Pattern)

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    MIGRATION ROADMAP (18 MONTHS)                         │
│                    STRANGLER FIG PATTERN                                 │
└──────────────────────────────────────────────────────────────────────────┘

PHASE 1: FOUNDATION (Months 1-3)
┌──────────────────────────────────────────────────────────────────────────┐
│  Activities:                                                             │
│  • AWS Landing Zone Setup                                                │
│  • Direct Connect to On-Premises                                         │
│  • MSK Cluster Deployment                                                │
│  • EKS Cluster Setup                                                     │
│  • API Gateway & WAF Configuration                                       │
│  • IAM & Cognito Setup                                                   │
│                                                                          │
│  Outcome: Integration platform ready for development                     │
│  Legacy:    Unchanged  |  New:    Building                               │
└──────────────────────────────────────────────────────────────────────────┘

PHASE 2: COEXISTENCE & EVENTIFICATION (Months 4-8)
┌──────────────────────────────────────────────────────────────────────────┐
│  Activities:                                                             │
│  • Debezium CDC on Legacy Core DB                                        │
│  • Kafka Connect for Legacy MQ                                           │
│  • Publish legacy events to MSK                                          │
│  • Build integration runtime for customer APIs                           │
│  • Connect new web/mobile channels                                       │
│  • Implement fraud detection on event stream                             │
│                                                                          │
│  Outcome: All systems publishing events; new channels live               │
│  Legacy:    Running (Publishing Events)  |  New:    Web/Mobile Live      │
└──────────────────────────────────────────────────────────────────────────┘

PHASE 3: STRANGLE - CUSTOMER MANAGEMENT (Months 9-12)
┌──────────────────────────────────────────────────────────────────────────┐
│  Activities:                                                             │
│  • Launch Digital Core Customer Domain                                   │
│  • Route customer APIs to Digital Core                                   │
│  • Implement dual-write with verification                                │
│  • Migrate customer data (batch + CDC)                                   │
```

```
│  • Decommission legacy customer interfaces                                  │
│  • Run parallel validation for 2 weeks                                      │
│                                                                             │
│  Outcome: Customer data mastered in Digital Core                            │
│  Legacy:    Customer Domain Strangled  |  New:    Customer Live             │
└─────────────────────────────────────────────────────────────────────────────┘

PHASE 4: STRANGLE - ACCOUNTS & TRANSACTIONS (Months 13-16)
┌─────────────────────────────────────────────────────────────────────────────┐
│  Activities:                                                                │
│  • Launch Digital Core Accounts Domain                                      │
│  • Migrate accounts in waves by product type                                │
│  • Implement transaction dual-processing                                    │
│  • Cutover transaction processing to Digital Core                           │
│  • Decommission legacy account interfaces                                   │
│                                                                             │
│  Outcome: Accounts and transactions in Digital Core                         │
│  Legacy:    Accounts Strangled  |  New:    Accounts Live                    │
└─────────────────────────────────────────────────────────────────────────────┘

PHASE 5: STRANGLE - LOANS & COMPLETION (Months 17-18)
┌─────────────────────────────────────────────────────────────────────────────┐
│  Activities:                                                                │
│  • Launch Digital Core Loans Domain                                         │
│  • Migrate loan portfolio                                                   │
│  • Decommission remaining legacy interfaces                                 │
│  • Final data reconciliation                                                │
│  • Decommission legacy core (read-only archive)                             │
│                                                                             │
│  Outcome: All capabilities migrated to Digital Core                         │
│  Legacy:    Decommissioned  |  New:    Fully Live                           │
└─────────────────────────────────────────────────────────────────────────────┘

┌─────────────────────────────────────────────────────────────────────────────┐
│                     ROLLBACK STRATEGY                                       │
│  Each migration wave includes:                                              │
│  • Feature flags for instant rollback                                       │
│  • Dark launches with traffic mirroring                                     │
│  • Automated reconciliation jobs                                            │
│  • Rollback procedure tested in pre-production                              │
│  • Maximum rollback time: 30 minutes                                        │
└─────────────────────────────────────────────────────────────────────────────┘
```

5. Technology Realization Decisions

5.1 Platform Selection Matrix

Category	Selected Technology	Alternatives	Decision Rationale
Cloud Provider	AWS	Azure, GCP	Market leader in financial services; broadest compliance (PCI, FedRAMP, SOC); used by Capital One, Goldman, Nasdaq
Container Orchestration	Amazon EKS	Self-managed K8s, ECS	Managed control plane; native AWS integration; industry standard

Category	Selected Technology	Alternatives	Decision Rationale
Event Streaming	Amazon MSK (Kafka)	Confluent, Kinesis	Kafka is banking standard (JPMorgan, Deutsche Bank); exactly-once semantics
API Management	AWS API Gateway	Kong, Apigee	Native AWS integration; serverless; WAF integration; lower TCO
Integration Runtime	Spring Boot on EKS	Quarkus, Node.js	Most widely adopted in banking; massive ecosystem; enterprise support
Data Streaming	Kafka Connect	AWS DMS	Standardized framework; vast connector ecosystem; exactly-once
Identity	Cognito + AD Connector	Auth0, Okta	Native AWS; cost-effective; supports customer + employee identities
Database	Amazon Aurora (PostgreSQL)	Oracle, SQL Server	PostgreSQL compatibility; 5x performance; Global Database; lower cost
Data Lake	Amazon S3	HDFS, Redshift	11 9's durability; infinite scalability; native analytics integration
Monitoring	CloudWatch + X-Ray + OpenSearch	Datadog, Splunk	Lower TCO; seamless integration; OpenSearch at 1/3 Splunk cost
Infrastructure as Code	Terraform	CloudFormation, CDK	Cloud-agnostic; multi-provider; large module ecosystem

5.2 Technology Justification Framework

Why AWS Over Azure/GCP?

Criterion	AWS	Azure	GCP
Financial Services Market Share	60%	25%	15%
Compliance Certifications	90+	70+	30+
Banking References	-	-	-

Decision: AWS provides the most mature, compliant, and widely-adopted cloud platform for global banking .

Why Kafka Over Alternatives?

Criterion	Kafka (MSK)	Kinesis	RabbitMQ
Exactly-Once Semantics			
Ordering Guarantee	(per partition)	(per shard)	
Retention (days)	Unlimited (S3)	7 days	Memory-bound
Replayability			
Throughput	1M+ msgs/sec	100K+ msgs/sec	10K+ msgs/sec
Banking Adoption	High	Medium	Low

Decision: Kafka's exactly-once semantics, ordering guarantees, and massive throughput make it the best choice for financial event streaming .

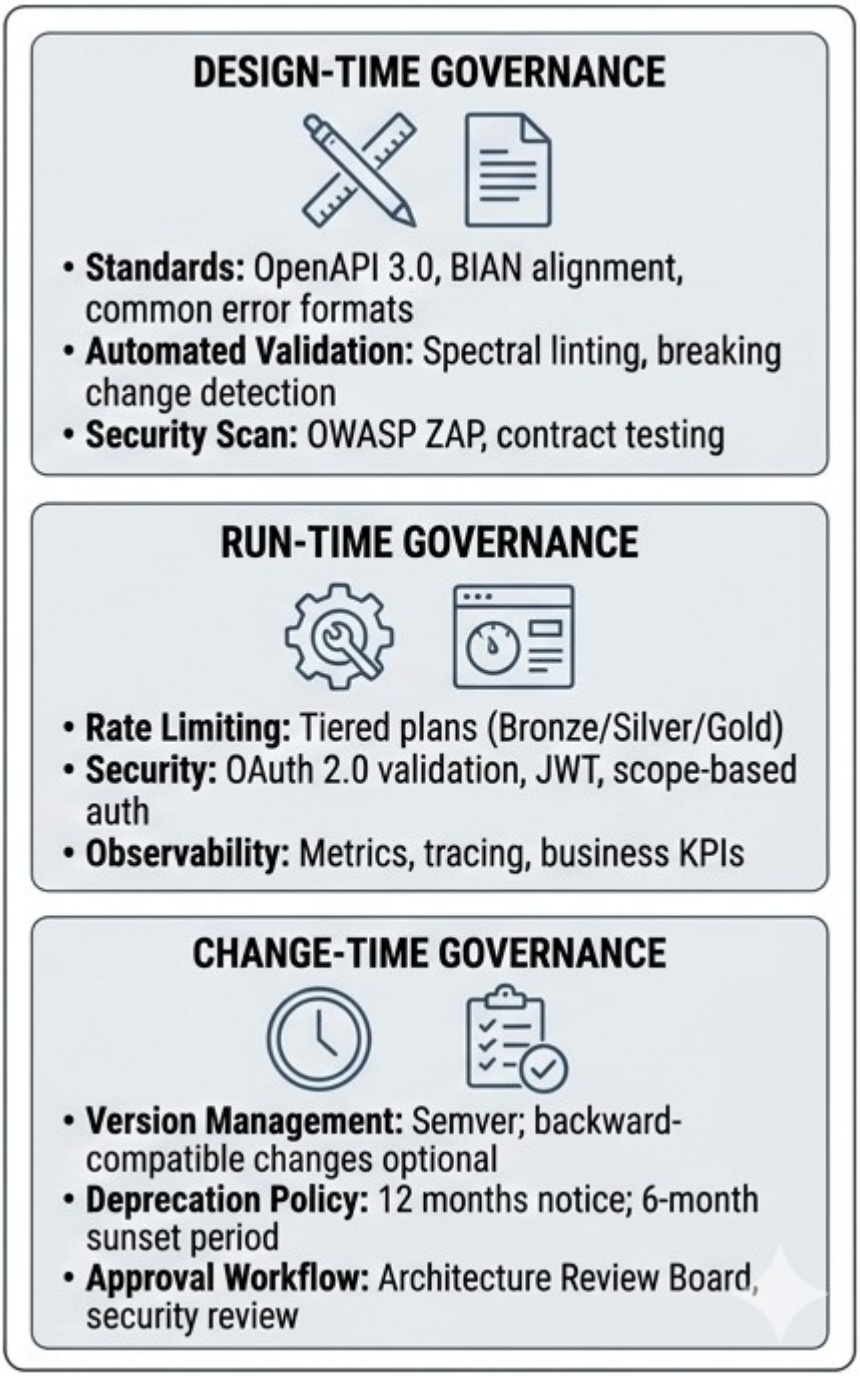
Why Spring Boot ?

Criterion	Spring Boot	Quarkus	Node.js
Banking Developer Availability	Very High	Medium	High
Enterprise Support	VMware	Red Hat	Community
Maturity	20+ years	5+ years	15+ years
Transaction Management	Excellent (JTA)	Good	Limited

Decision: Spring Boot's maturity, enterprise support, and transaction capabilities align perfectly with banking requirements.

6. Governance & Operating Model

6.1 API Governance Framework



6.2 Center for Enablement (C4E) Model

```
┌──────────────────────────────────────────────────────────────────────┐
│              CENTER FOR ENABLEMENT (C4E) OPERATING MODEL              │
│                                                                       │
│  ┌──────────────────────────────────────────────────────────────────┐
│  │ PLATFORM TEAM (Inner Circle)                                      │
│  │ • Core platform infrastructure (AWS, EKS, MSK)                    │
│  │ • Shared services (API Gateway, Cognito)                          │
│  │ • Base images and CI/CD pipelines                                 │
│  │ • Security standards and compliance                               │
│  │ • Developer experience and tooling                                │
│  │ Team Size: 8-10 (Platform Engineers, SREs, Security Architects)   │
│  └──────────────────────────────────────────────────────────────────┘
│                                                                       │
│  ┌──────────────────────────────────────────────────────────────────┐
│  │ DOMAIN TEAMS (Outer Circle)                                       │
│  │ • Customer Domain Team                                            │
│  │ • Accounts Domain Team                                            │
│  │ • Payments Domain Team                                            │
│  │ • Lending Domain Team                                             │
│  │ • Fraud & Risk Domain Team                                        │
│  │ Team Size per Domain: 5-7 (Developers, QA, Domain Architect)      │
│  └──────────────────────────────────────────────────────────────────┘
└──────────────────────────────────────────────────────────────────────┘
```

6.3 Microservices Governance

Aspect	Standard	Enforcement
Service Size	2-6 weeks of development	Architecture review
Database per Service	Each service owns its schema	CI/CD pipeline check
Communication	REST for sync, Kafka for async	Design review
Resilience	Circuit breakers, retries, timeouts	Code review + chaos engineering
Observability	Health checks, metrics, logs, traces	Automated in platform
Documentation	OpenAPI spec, README, runbooks	Mandatory in PR template

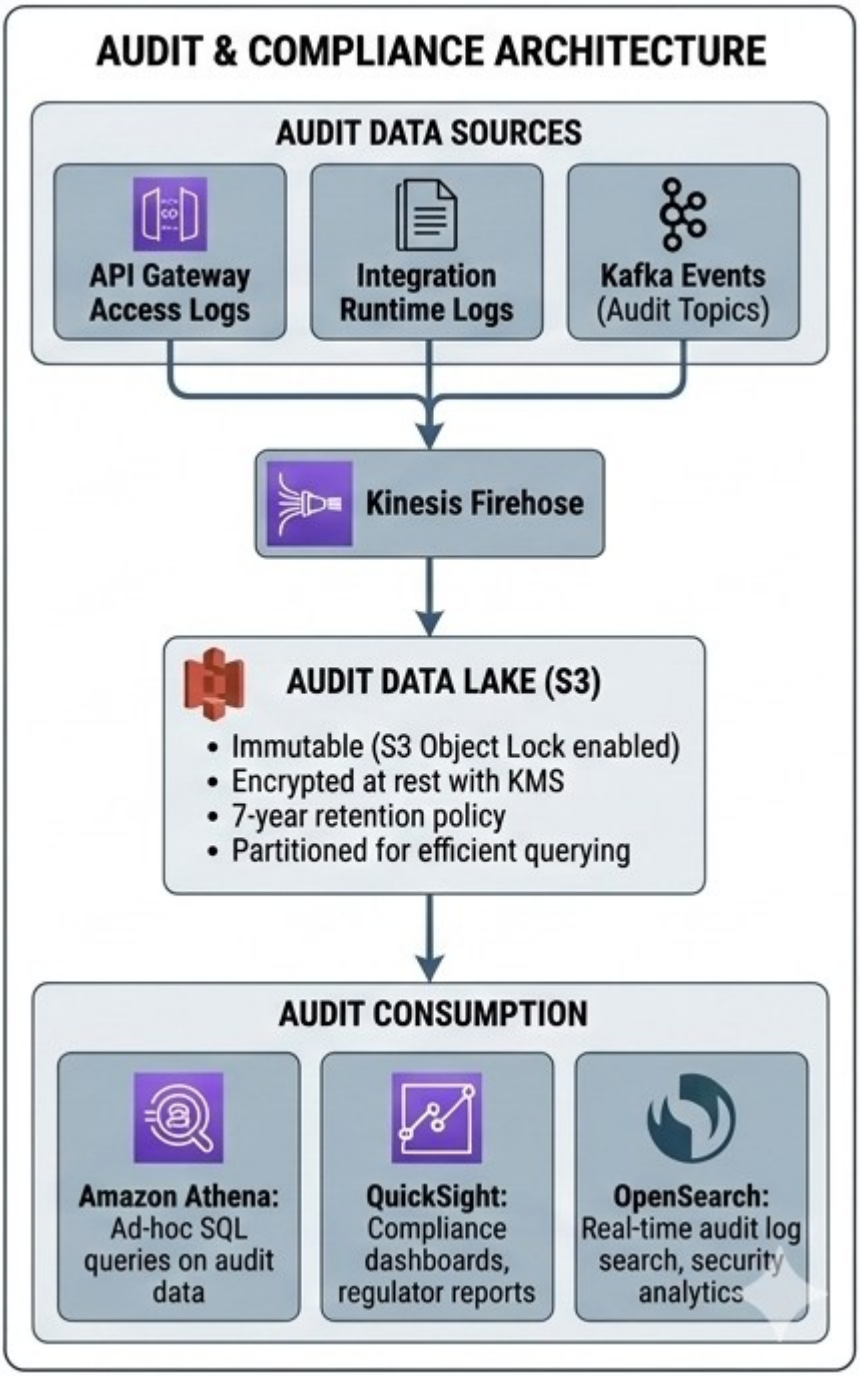
7. Compliance & Regulatory Framework

7.1 Global Regulatory Mapping

Regulation	Requirement	Implementation
PCI-DSS	Protect cardholder data	Tokenization; KMS HSM; TLS 1.3; fine-grained IAM
GDPR	Right to erasure; consent management	Data deletion API; consent service with audit trail

Regulation	Requirement	Implementation
SOX	Financial reporting controls	Exactly-once event processing; reconciliation jobs
Basel III	Risk data aggregation	Real-time risk event streams; data lineage
PSD3/PSR	Strong authentication; open banking APIs	OAuth 2.0/OIDC; consent management; API standards
Local Regulations	Data residency; local reporting	Regional data storage; automated regulator reports

7.2 Audit Architecture



7.3 Data Privacy & Protection

Classification	Examples	Storage	Encryption	Access	Retention
Public	Product rates	S3 public	TLS only	Unauthenticated	Indefinite
Internal	Operational metrics	Platform	At rest + TLS	Employee	90 days
Confidential	Customer names, addresses	Tokenized	KMS + TLS	Need-to-know	7 years
Restricted	PAN, SSN, account numbers	Tokenized	KMS HSM + TLS	Limited roles	7 years
Regulatory	Audit logs	S3 Object Lock	KMS + TLS	Auditors	7-10 years

8. Appendix

8.1 BIAN Service Domain Mapping

BIAN Service Domain	API Gateway Endpoint	Core System	Event Topic
Party Reference Data	`/api/customer-management`	Digital Core	`customer.events`
Current Account	`/api/accounts/current`	Digital Core	`account.events`
Savings Account	`/api/accounts/savings`	Digital Core	`account.events`
Payment Execution	`/api/payments`	Payment Platform	`transaction.events`
Loan	`/api/loans`	Digital Core	`loan.events`
Fraud Detection	Internal	Fraud System	`fraud.alerts`

8.2 API Performance SLAs

API Category	P95 Latency	P99 Latency	Availability
Customer Information	< 200ms	< 500ms	99.99%
Account Balance	< 100ms	< 300ms	99.99%
Transaction History	< 500ms	< 1s	99.95%
Payment Initiation	< 1s	< 2s	99.99%
Loan Application	< 2s	< 5s	99.95%
Open Finance APIs	< 500ms	< 1s	99.9%

8.3 Disaster Recovery Playbook Summary

Scenario	Detection	Action	RTO	RPO
Single AZ Failure	CloudWatch alarm	Auto-healing by K8s/MSK	< 1m	0

Scenario	Detection	Action	RTO	RPO
Single Region Failure	Route53 health check	DNS failover to secondary	< 5m	< 1s
Data Corruption	Reconciliation alert	Point-in-time recovery	< 30m	5m
Full Site Failure	Synthetic monitoring	Activate DR region	< 5m	< 1s

8.4 Glossary

Term	Definition
BIAN	Banking Industry Architecture Network - standard service domains for banking
EDA	Event-Driven Architecture - architectural pattern based on event production/consumption
MSK	Amazon Managed Streaming for Apache Kafka
EKS	Amazon Elastic Kubernetes Service
Strangler Pattern	Migration pattern that gradually replaces legacy components
RPO	Recovery Point Objective - maximum acceptable data loss
RTO	Recovery Time Objective - maximum acceptable downtime
mTLS	Mutual Transport Layer Security - two-way certificate-based authentication
CDC	Change Data Capture - technology for capturing database changes in real-time